
Entwicklung einer Webapplikation zur Unterstützung der Terminplanung von Projekten

Bachelorarbeit zur Erlangung des akademischen Grades Bachelor of Science

Studiengang Informatik – Software und Information Engineering



Betreut von

Mag. Dr. Peter Reiter

Vorgelegt von

Timo Emanuel Rzipa

Dornbirn, 19.07.2018

Geschlechtergerechte Sprache

Der Verfasser der vorliegenden Arbeit bekennt sich zu einer geschlechtergerechten Sprachverwendung. Um die Lesbarkeit zu gewährleisten und zugunsten der Textökonomie, werden die verwendeten Personen bzw. Personengruppen fix männlich oder weiblich zugeordnet. Zum Beispiel werden immer „die Softwareentwicklerin“ und „der Kunde“ verwendet. Es wurde besonders darauf geachtet, stereotype Rollenbeschreibungen zu vermeiden. Die insgesamt eventuell dadurch hervorgerufene Irritation bei den Lesenden ist gewünscht und soll dazu beitragen, eine Bewusstheit für die bestehende, Frauen diskriminierende Sprachgewohnheit (generelle Verwendung der männlichen Begriffe für beide Geschlechter) zu wecken bzw. zu stärken.

Eidesstattliche Erklärung

Ich erkläre hiermit ehrenwörtlich, dass ich die vorliegende Arbeit selbstständig angefertigt habe. Die aus fremden Quellen direkt oder indirekt übernommenen Gedanken sind als solche kenntlich gemacht. Die Arbeit wurde bisher keiner anderen Prüfungsbehörde vorgelegt und auch noch nicht veröffentlicht.

Kurzreferat

Es soll eine Webapplikation entwickelt werden, welche die Terminplanung in Projekten unterstützen soll. Dafür soll es die Möglichkeit geben, PDM (Precedence Diagramming Method) Netzwerke grafisch modellieren zu können. Ergebnisse verschiedener Analysen wie CPM (Critical Path Method) oder PERT (Program Evaluation and Review Technique), welche auf einem Server berechnet werden, sollen in der Webapplikation übersichtlich dargestellt werden. Für die Entwicklung des Rest-Servers wird Java verwendet und für den Client wird auf JavaScript zurückgegriffen.

Als Erstes wurden der Rest-Service und die Webapplikation erstellt und sichergestellt, dass die Kommunikation funktioniert. Anschließend wurde in der Webapplikation das Zeichnen der Pläne ermöglicht, welche anschließend an den Server gesendet werden. Auf dem Server wurden dann unterschiedliche Analysen wie CPM, PERT und die Monte-Carlo-Simulation implementiert. Schlussendlich wurde in der Webapplikation die visuelle Darstellung der Ergebnisse der Analysen umgesetzt.

Im Laufe der Arbeit konnten alle geplanten Features implementiert werden. Die Webapplikation ermöglicht das Zeichnen der Pläne, kann mit Hilfe des Rest-Services Analysen durchführen, und diese auch visuell darstellen.

Abstract

A web-application should be developed, which should support the scheduling of a project. For this purpose, there should be the possibility to draw PDM (Precedence Diagramming Method) networks. Results of various analyses like CPM (Critical Path Method) or PERT (Program Evaluation and Review Technique), which are calculated on a server, should be displayed clearly in the web application. Java is used to develop the Rest-server and JavaScript is used for the client.

First of all, the Rest-service and the web application were created, then it was ensured that the communication works. Subsequently, drawing the plans in the web application was made possible. Those diagrams are then sent to the server. Different evaluation methods like CPM, PERT or the Monte-Carlo-Simulation were then implemented on the server. Finally, the visual presentation of the results of the evaluations was implemented in the web application.

During the work, all planned features could be implemented in time. The web application allows the drawing of plans, analyses them with the help of the rest service and also displays those results visually.

Inhaltsverzeichnis

1. Einführung	1
1.1. Motivation.....	1
1.2. Aufgabenstellung/Ziele	2
1.3. PDM – Precedence Diagram Method	3
1.4. CPM – Critical Path Method.....	4
1.5. PERT – Program Evaluation and Review Technique	6
1.6. Monte-Carlo-Simulation.....	7
2. Stand der Technik.....	8
3. Technologien.....	9
3.1. Java	9
3.2. Rest - Representational State Transfer.....	9
3.3. HTML5 & CSS	10
3.4. JavaScript	10
3.5. Postman	10
3.6. Frameworks & Bibliotheken	11
4. Aufbau	13
4.1. Webservice.....	13
4.2. Rest-Service.....	15
5. Umsetzung	17
5.1. Recherche.....	17
5.2. Rest-Service.....	18
5.3. Webapplikation	23
6. Ergebnisse	25
6.1. Webapplikation	25
6.2. Rest-Service.....	28

6.3.	Ausblick/Erkenntnisse.....	28
7.	Literaturverzeichnis.....	29
8.	Abbildungsverzeichnis.....	31

1. Einführung

Bei dieser Arbeit handelt es sich um die Abschlussarbeit des Studienganges Informatik – Software and Information Engineering an der FH Vorarlberg zur Erlangung des akademischen Grades Bachelor of Science. Dabei wurde das Thema von Mag. Dr. Peter Reiter vorgegeben und auch durch ihn betreut.

1.1. Motivation

Um ein Projekt leiten zu können, müssen im Vorhinein und während dem Projekt immer wieder Pläne oder Diagramme erstellt bzw. überarbeitet werden. Diese Arbeit soll dabei helfen, zumindest einen Teil dieser Tätigkeiten teilweise zu automatisieren und zu erleichtern. Dabei soll ein Tool entwickelt werden, welches Analysen wie CPM (Critical Path Method), PERT (Program Evaluation and Review Technique) oder die Monte Carlo Simulation, durchführen kann. Durch die CPM kann dabei der kritische Pfad bestimmt werden. Jede Aktivität welche auf dem kritischen Pfad liegt, verzögert das gesamte Projekt. Mittels PERT kann durch drei Zeitangaben, der Minstdauer, der erwarteten Dauer und der Maximaldauer eine Gesamtdauer berechnet werden. Bei der Monte Carlo Simulation werden Zufallsexperimente auf Basis der drei Zeitangaben durchgeführt, was die Berechnung der Varianz oder der Wahrscheinlichkeit für ein Element auf dem kritischen Pfad zu liegen ermöglicht. Somit werden mehrere verschiedene Methoden in einer Anwendung vereint. Die genauen Vorteile der einzelnen Methoden können jeweils den folgenden Kapiteln entnommen werden.

1.2. Aufgabenstellung/Ziele

Ziel dieser Bachelorarbeit ist es, eine Webapplikation zu entwickeln, welche es erlaubt, PDM (Precedence Diagram Method)-Netzwerke grafisch darzustellen. Zudem sollen Ergebnisse verschiedener Analysen grafisch dargestellt werden, es soll zum Beispiel der kritische Pfad markiert werden. Dabei werden verschiedene Techniken wie CPM, PERT oder die Monte Carlo Simulation eingesetzt. Da die Monte Carlo Simulation sehr rechenintensiv sein kann, soll ein Rest-Service implementiert werden, welcher die Berechnungen durchführt und die Ergebnisse zurück an die Webapplikation sendet, wo diese grafisch dargestellt werden.

Für die Entwicklung des Servers soll Java verwendet werden und beim Client soll auf JavaScript gesetzt werden. Bibliotheken und Frameworks können frei gewählt werden, allerdings muss die Anwendung ohne Lizenzfolgekostenfolgen wart- und benutzbar sein.

1.3. PDM – Precedence Diagram Method

PDM, auf Deutsch auch oft Netzplantechnik genannt, definiert eine Art und Weise, wie Aktivitäten in einer Abfolge darzustellen sind. Diese Methode wird meistens in Kombination mit CPM oder PERT verwendet. Die Aktivitäten, welche in diesem Diagramm dargestellt werden, kommen meistens aus dem PSP (Projektstrukturplan). Je nach Projektumfang können die Aktivitäten aus dem PSP auch in mehrere Sub-Aktivitäten zerlegt werden. Abbildung 1 stellt ein simples PDM-Netzwerk dar, welches einen Start- und Endknoten beinhaltet, welche nicht zwingend notwendig sind, bei mehreren Start- und Endknoten aber zur Übersichtlichkeit beitragen. A, C und D sind die Startelemente, welche alle gleichzeitig gestartet werden. Anschließend werden alle darauffolgenden Aktivitäten bearbeitet, so lange bis alle abgeschlossen sind. Dabei können Aktivitäten, wie B, nur starten, wenn alle vorangegangenen Aktivitäten abgeschlossen sind. (Nahmias; Olsen 2015)

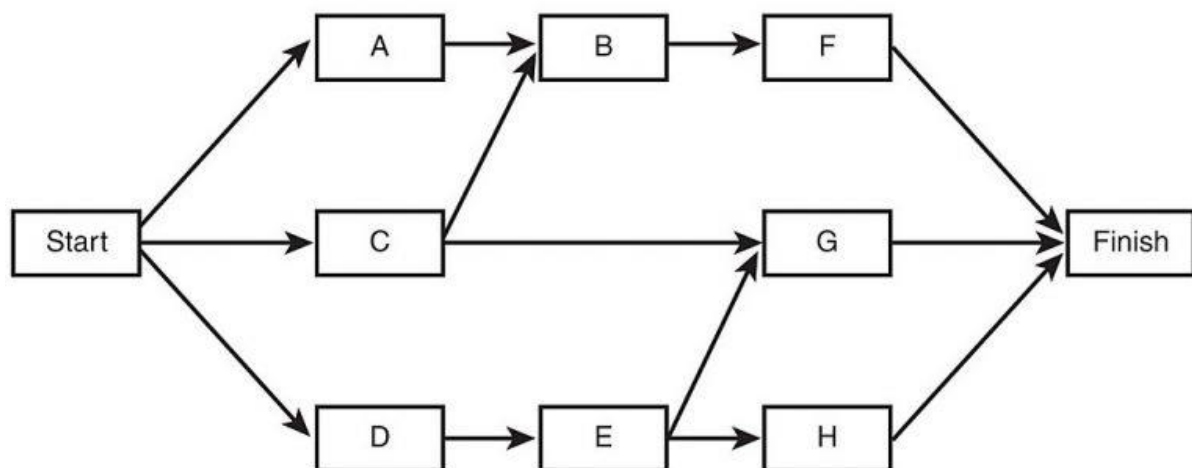


Abbildung 1: PDM-Netzwerk („PDM-1-768x328.jpg (768x328)“ o. J.)

1.4. CPM – Critical Path Method

Bei CPM handelt es sich um eine Methode, welche alle kritischen Pfade in einem PDM-Netzwerk ermittelt. Alle Aktivitäten, welche auf dem kritischen Pfad liegen, verzögern, falls diese länger als geplant dauern, das gesamte Projekt. Dabei werden nicht nur die kritischen Pfade gespeichert, sondern es wird für jede Aktivität das früheste Start- und Enddatum, sowie das späteste Start- und Enddatum gespeichert. Dabei müssen folgenden Regeln beachtet werden:

- Eine Aktivität kann nur beginnen, wenn alle vorhergehenden Aktivitäten abgeschlossen sind.
- Kann eine Aktivität beginnen, bevor die vorangegangene Aktivität komplett abgeschlossen ist, so muss die vorangegangene Aktivität unterteilt werden.
- Jede Aktivität darf nur einmal ablaufen, es dürfen keine Schleifen entstehen.

Die Berechnung des kritischen Pfades erfolgt in zwei Schritten:

Im ersten Schritt, der Vorwärtsterminierung, werden von der Startaktivität/den Startaktivitäten aus alle Aktivitäten und somit auch alle möglichen Pfade durchlaufen. Dabei ergeben sich die frühesten Start- und Endtermine. Beim kritischen Pfad handelt es sich um alle Pfade, welche die höchste Gesamtdauer aufweisen. Verzögert sich eine Aktivität, welche in einem kritischen Pfad enthalten ist, so verzögert sich damit das gesamte Projekt.

Im zweiten Schritt, der Rückwärtsterminierung, werden von der Endaktivität/den Endaktivitäten aus alle Aktivitäten und somit auch alle möglichen Pfade durchlaufen. Dabei ergeben sich die spätesten Start- und Endtermine. Mit Hilfe dieser Daten kann man nun feststellen, wie viel später man maximal mit einer Aktivität anfangen kann, welche nicht im kritischen Pfad enthalten ist. (Nahmias; Olsen 2015)

1.4.1. Berechnung des kritischen Pfades & Bestimmung Startzeitpunkte

In Abbildung 2 wird ein einfaches Beispiel dargestellt. Dabei steht links oben die früheste Startzeit, oben mittig die Dauer und oben rechts die früheste Endzeit. Darunter steht der Name der Aktivität, unten links die späteste Startzeit und unten rechts die späteste Endzeit.

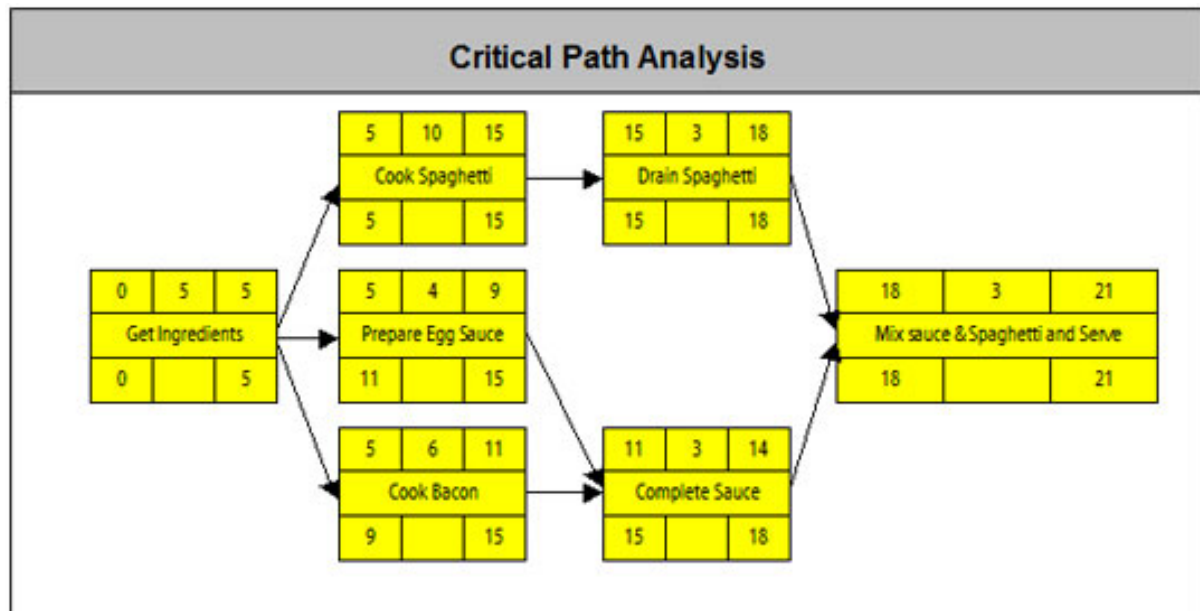


Abbildung 2: Beispiel CPM

Bei der ersten Aktivität ist die Berechnung unkompliziert, der Startzeitpunkt ist 0, plus die Dauer von 5 ergibt früheste und späteste Endzeit. Diese Endzeit stellt gleichzeitig die früheste Startzeit für alle folgenden Aktivitäten dar. Das Addieren der Dauer ergibt wieder die früheste Endzeit. Hat eine Aktivität zwei oder mehr vorangegangene Aktivitäten, wie z.B. „Complete Sauce“ dann muss die höhere Dauer, in diesem Fall „11“ übernommen werden. Am Ende angekommen, wird nun in die andere Richtung gerechnet. Die späteste Startzeit der Endaktivitäten wird auf die vorangegangenen Aktivitäten übernommen. Abzüglich der Dauer kann nun der späteste Startzeitpunkt berechnet werden, welcher dann wiederum für die vorangegangenen Aktivitäten übernommen wird. Hat eine Aktivität zwei oder mehr Folgeaktivitäten wird die niedrigste späteste Startzeit übernommen, wie z.B. bei „Get Ingredients“. Der kritische Pfad besteht aus allen Elementen, welche keinen Puffer haben, also alle Elemente, welche bei der frühesten und spätesten Startzeit und bei der frühesten und spätesten Endzeit dieselben Werte aufweisen. In diesem Fall wären das „Get Ingredients“, „Cook Spaghetti“, „Drain Spaghetti“ und „Mix sauce & Spaghetti and Serve“.

1.5. PERT – Program Evaluation and Review Technique

PERT wurde 1958 von der Navy entwickelt, um eine Rakete zu entwickeln, welche es so zuvor noch nie gegeben hat. Da niemand den Zeitaufwand für Forschung oder Entwicklung auch nur annähernd einschätzen konnte, wurde PERT entwickelt und angewendet. Dabei wurde das Projekt Schätzungen zufolge Dank der Verwendung von PERT bis zu zwei Jahre früher fertiggestellt. (United States; Bureau of Naval Weapons; Special Projects Office 1958)

Im Gegensatz zu CPM, wird PERT bei sehr hoher Unsicherheit und sehr wenig Erfahrung verwendet. Es wird nicht wie bei CPM eine Dauer festgelegt, sondern eine minimale (d_{min}), eine erwartete (d_{norm}) und eine maximale (d_{max}) Dauer. Aus diesen drei Werten wird anschließend ausgehend von einer Beta-Verteilung über folgende Formel eine mittlere Dauer (d_{mittel}) berechnet:

$$d_{mittel} = \frac{d_{min} + 4 \cdot d_{norm} + d_{max}}{6}$$

Abbildung 3: Formel für mittlere Dauer

Die Beta-Verteilung hängt von den drei festgelegten Werten ab. Diese könnte zum Beispiel folgendermaßen aussehen:

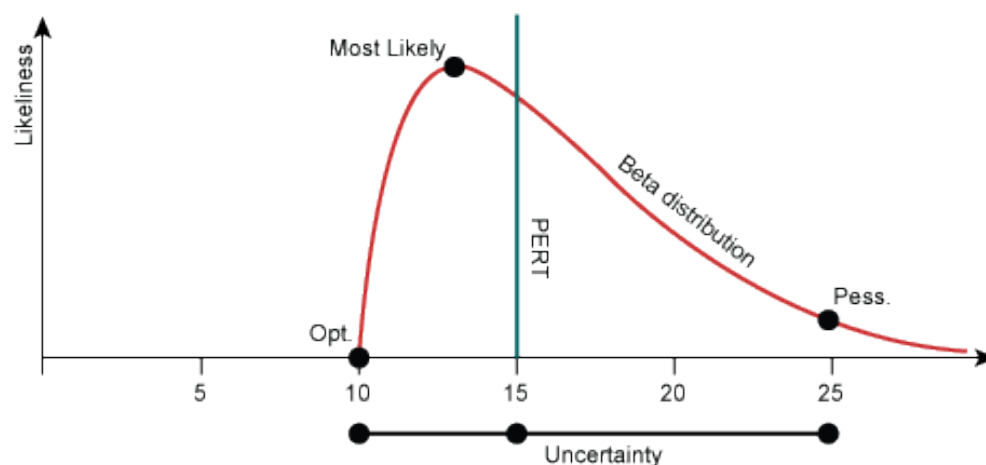


Abbildung 4: Beta-Verteilung (DavidJWilliamson 2016)

Bei PERT wäre es außerdem möglich, verschiedene Entscheidungsknoten, wie UND, ODER oder EXLUSIVES-ODER zu verwenden. Da in dieser Arbeit aber CPM und PERT kombiniert werden, wurden diese Entscheidungsknoten nicht berücksichtigt. (Nahmias; Olsen 2015)

Min time	Est time	Max time	Dur
2	4	6	

→

Min time	Est time	Max time	Dur
2	4	6	4

Abbildung 5: Benötigte Werte für PERT und Ergebnis

1.6. Monte-Carlo-Simulation

Bei der MC-Simulation handelt es sich um ein Stochastisches Verfahren, welches eine große Anzahl gleichartiger Zufallsexperimente durchführt. Die Grundlage für diese Simulation ist das Gesetz der großen Zahlen. Die MC-Simulation wird oft verwendet, um sehr komplexe Probleme zu lösen.

Bei dieser Arbeit soll die Monte Carlo Simulation verwendet werden, um eine Alternative zu PERT zu bieten. Im Gegensatz zu PERT, welches einen fixen Wert ausrechnet, wird bei der MC-Simulation eine gewisse Anzahl von Zufallsexperimenten durchgeführt. Als Basis für die Zufallsexperimente dient die Beta-Verteilung. Dadurch kann für jede Aktivität ein Zufallsexperiment durchgeführt werden, welches eine Dauer als Ergebnis liefert. Anschließend wird für diese Zufallswerte die CP-Methode durchgeführt, welche den kritischen Pfad für diese Konstellation evaluiert. All dies wird X-Mal durchgeführt wobei alle Ergebnisse gespeichert werden.

Mithilfe der Ergebnisse der Zufallsexperimente lassen sich nun folgende Werte berechnen:

- P (Aktivität auf kritischen Pfad)
- Varianz der Dauer
- Dauer (Mittlerwert aller Zufallsexperimente)

Dabei ist es möglich, dass das Endergebnis mit dem von PERT übereinstimmt, es kann jedoch auch davon abweichen. Da die MC-Simulation auf Zufallsexperimenten basiert, ist es möglich, dass bei mehrmaligen Berechnungen unterschiedliche Ergebnisse gefunden werden. (Rubinstein; Kroese 2007)

2. Stand der Technik

Es gibt nur sehr wenige Tools, welche auf das Erstellen von PDM-Netzwerken spezialisiert sind. Es ist möglich mit Visio oder anderen Zeichenprogrammen solche Netzwerke zu zeichnen, jedoch können diese Programme nicht miteinander verglichen werden, denn die Webapplikation soll zusätzlich zum Zeichnen auch Analysen durchführen können.

Die meisten Programme welche Analysen wie CPM oder PERT durchführen können, sind über zehn Jahre alt und verwenden zur Darstellung meist ein Gantt-Diagramm und kein PDM-Netzwerk. Microsoft Project, zum Beispiel, erlaubt die Berechnung des kritischen Pfades sowie das Verwenden von PERT, allerdings kann dies wiederum nur als Gantt-Diagramm dargestellt werden. Ebenfalls gibt es einige Excel-Add-ins (z.B. OpenPERT), welche Methoden wie PERT und/oder CPM unterstützen, wobei hier wiederum die Darstellung als PDM fehlt. Die Add-ins, welche eine graphische Darstellung ermöglichen, bieten nur Gantt-Diagramme an.

Es gibt einige Online-Tools, welche Analysen wie CPM oder PERT unterstützen, z.B. Lucidchart („Online Diagram Software & Visual Solution“ 2018), LiquidPlanner (Liquidplanner o. J.) oder GanttProject („GanttProject: free desktop project management app“ o. J.), allerdings konnte kein Tool gefunden werden, welches die Erstellung von PDM-Netzwerken in Kombination mit Analysen wie CPM, PERT oder Monte Carlo Simulation anbietet. Außerdem ist die Mehrzahl der Online-Tools nicht kostenlos verwendbar.

3. Technologien

In diesem Kapitel werden zuerst die verwendeten Technologien, Anwendungen und Frameworks beschrieben und erklärt, warum diese verwendet wurden. Danach werden Alternativen beschrieben und erläutert, warum diese nicht verwendet wurden.

3.1. Java

Java ist eine objektorientierte Programmiersprache welche mit dem Ziel entwickelt wurde, CPU und plattformunabhängig zu sein. Heute ist Java weit verbreitet und läuft auf mehr als drei Milliarden Geräten. Die aktuelle Version 10 ist zum Zeitpunkt des Schreibens erst wenige Monate alt, weswegen Version 9 zur Implementierung des Rest-Services verwendet wurde. (Niemann 2010)

3.2. Rest - Representational State Transfer

Rest ist ein Schema, welches für verteilte Systeme, insbesondere für Webservices, verwendet wird. Rest dient der Kommunikation zwischen dem Client (der Webapplikation) und dem Server (Rest-Service). Die Kommunikation erfolgt über HTTP, wobei die Daten meist als JSON (JavaScript Object Notation) oder XML (Extensible Markup Language) versendet werden. Der Vorteil von Rest gegenüber anderen Systemen wie SOAP (Simple Object Access Protocol), ist die verhältnismäßige Einfachheit, da eine einheitliche Schnittstelle verwendet wird. Rest bietet sich als einfachere Alternative zu ähnlichen Schemen wie SOAP oder WSDL (Web Services Description Language) an.

Rest ist ein zustandsloses System, was bedeutet, dass alle Informationen, welche benötigt werden, in die Rest-Nachricht gepackt werden müssen. Es dürfen keine Zustandsinformationen zwischen den Nachrichten gespeichert werden. Diese Zustandslosigkeit begünstigt die Skalierbarkeit des Webservices, da eingehende Anfragen einfach auf beliebige Maschinen verteilt werden können, da jede Anfrage in sich geschlossen ist und auf dem Server keinerlei Zustandsinformationen gespeichert werden.

Durch die Verwendung der HTTP-Methoden GET, POST, PUT und DELETE können alle Schnittstellen, vorausgesetzt ein passender Name wird verwendet, selbsterklärend sein. (Tilkov u.a. 2015)

3.3. HTML5 & CSS

HTML5 ist die aktuellste Version von HTML, welche zusammen mit CSS verwendet wird um Webseiten darzustellen. HTML kümmert sich dabei um die Struktur und Funktionen von Elementen, wobei CSS das Aussehen dieser definiert. (Pilgrim 2010)

3.4. JavaScript

Bei JavaScript handelt es sich um die Programmiersprache des World Wide Webs. Durch die 1995 vorgestellte Programmiersprache, können Webseiten um dynamische Funktionen erweitert werden. Nur durch die Einbindung von JavaScript kann eine Website dynamisch auf Benutzereingaben reagieren. (Haverbeke 2015)

3.5. Postman

Postman (<https://www.getpostman.com>) ist ein Tool, mit dessen Hilfe man sehr einfach API-Schnittstellen testen kann. Es bietet die Möglichkeit, diverse HTTP-Anfragen zu senden, wobei die Autorisierung, der Header oder der Body angepasst werden können. So kann z.B. zum Testen der API eine Anfrage erstellt werden, welche dann immer wieder an den Server gesendet wird, bis das Ergebnis, welches ebenfalls von Postman dargestellt wird, befriedigend ist. („Postman“ o. J.)

3.6. Frameworks & Bibliotheken

3.6.1. Spring Boot

Durch die Verwendung von Spring Boot (<https://projects.spring.io/spring-boot/>), ist es möglich in kürzester Zeit verschiedene Services, wie z.B. Rest, aufzusetzen. Durch die Einbindung der richtigen Abhängigkeiten werden alle benötigten Funktionen einfach eingebunden. Anschließend kann in wenigen Schritten ein lauffähiger Rest-Service erstellt werden, dafür muss nur eine Klasse implementiert werden. Die benötigten Schritte sind dabei auf der offiziellen Website aufgelistet und können einfach abgearbeitet werden.

Spring übernimmt eine Vielzahl an Aufgaben, wie die Serialisierung und De-Serialisierung der HTTP-Antworten bzw. Anfragen. („Building a RESTful Web Service“ o. J.) Durch die Verwendung von Spring Boot ist es außerdem sehr leicht möglich, den Service in einem Docker-Container auszuführen, was weitere Arbeiten und Weiterentwicklungen stark vereinfachen würde. („Spring Boot Development with Docker“ 2017)

3.6.2. JointJS

JointJS (<https://www.jointjs.com/>) ist eine JavaScript-Bibliothek, welche das Erstellen und Bearbeiten verschiedener Diagramme ermöglicht. Die Möglichkeit mittels HTML sehr einfach eigene Diagramm-Elemente zu erstellen, hatte dabei den größten Einfluss auf die Auswahl dieser Bibliothek. Denn für das Erstellen eines PDM-Diagramms, welches CPM, PERT und die MC-Simulation kombiniert, musste ein Element erstellt werden, welches alle benötigten Ein- und Ausgabefelder kombiniert.

JointJS bietet weitere nützliche Features, wie eine einfache Serialisierung und De-Serialisierung, was gerade in Bezug auf die Verwendung von Rest von Vorteil ist. Ebenfalls werden eine Touch-Unterstützung sowie eine Zoom-Unterstützung angeboten. („JointJS: Visualize and interact with diagrams and graphs“ o. J.)

3.6.3. JQuery

Bei JQuery (<https://jquery.com/>) handelt es sich um eine JavaScript-Bibliothek, welche von JointJS verwendet wird. JQuery vereinfacht viele Funktionen, wie z.B. die Bearbeitung von Ereignissen, Animationen oder Ajax-Anfragen, welche für die Asynchrone Kommunikation mit dem Rest-Service benötigt werden. (jquery.org o. J.)

3.6.4. React Diagrams

React Diagrams ist eine Bibliothek für React, welche es ermöglicht, schnell und einfach Diagramme zu erstellen. Diese ist relativ einfach zu verwenden und dabei sehr performant. Allerdings stellte sich während der Recherche heraus, dass es keine Dokumentation gibt, welche beschreibt, wie eigene Elemente erstellt werden. Da das Erstellen eigener Elemente essentiell für die Umsetzung des PDM-Diagramms ist, wurde diese Bibliothek relativ früh verworfen. Ebenfalls würden die meisten Funktionen der Bibliothek ohnehin nicht benötigt werden, weshalb eine Verwendung auch aus diesem Grund nicht sehr sinnvoll gewesen wäre. („Introduction · react-diagrams“ o. J.; „react-diagrams: a super simple, no-nonsense diagramming library written in react that just works“ 2018)

4. Aufbau

Damit die Webapplikation funktioniert, muss der Server zwei Services anbieten:

- Den Webservice, mit welchem der Client die Website laden kann.
- Den Rest-Service, welcher für die Berechnungen benötigt wird.

In Abbildung 6 wird dargestellt, wie der Client im 1. Schritt die Website vom Webservice anfordert. Nach vollständigem laden der Website kann der Client die Funktionen des Rest-Services in Anspruch nehmen.

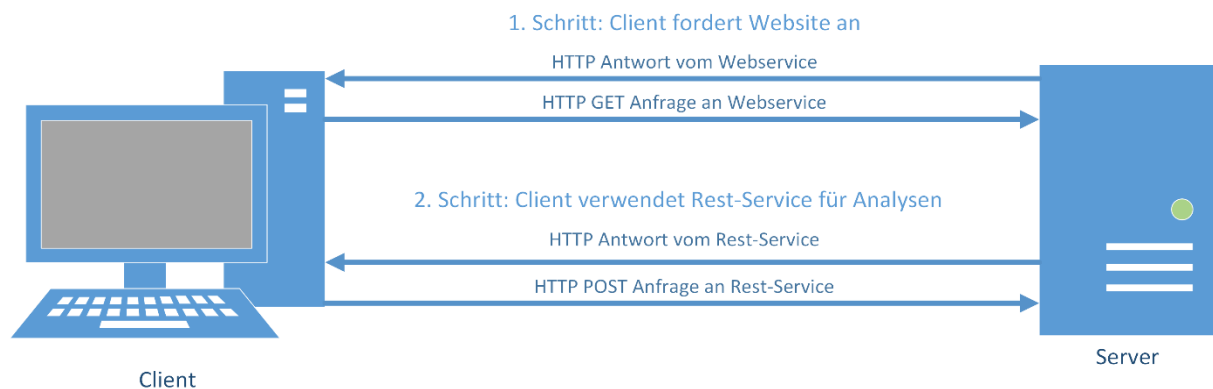


Abbildung 6: Client-Server Architektur

4.1. Webservice

Der Webservice muss nur eine einfache HTML-Datei anbieten, welche von einigen JavaScript-Dateien und einer CSS-Datei vervollständigt wird. Der Webservice muss somit kein PHP oder ähnliche Funktionen unterstützen. Es wäre sogar möglich die HTML-Datei direkt aus dem Datei-Browser heraus zu starten, gäbe es nicht das Problem der Same-Origin-Policy (SOP), welches in Kapitel 4.1.1 genauer beschrieben wird. Für diese Arbeit soll der IIS („Home : The Official Microsoft IIS Site“ o. J.), der Standard Microsoft Webservice verwendet werden, welcher alle benötigten Funktionen anbietet. Es wäre ebenfalls möglich das sehr beliebte XAMPP-Programmpaket von Apache zu verwenden, welches ebenfalls alle benötigten Funktionen anbietet, dabei aber auch weitere Features wie z.B. eine Datenbank zur Verfügung stellt. („XAMPP Installers and Downloads for Apache Friends“ o. J.)

4.1.1. SOP – Same-Origin-Policy

Bei der Same-Origin-Policy handelt es sich um ein Sicherheitskonzept, welches Skriptsprachen wie JavaScript untersagt, auf Objekte zuzugreifen, welche von einer anderen Website stammen. Der Rest-Service kann zwar auf demselben Server wie der Webservice ausgeführt werden, er muss jedoch auf einem anderen Port laufen. Aus diesem Grund wird der Rest-Service vom Browser als eine neue Origin wahrgenommen, was dazu führt, dass alle Anfragen an diesen blockiert werden. Dieses Problem kann behoben werden, indem das sogenannte Cross-Origin Resource Sharing (CORS) serverseitig implementiert wird. Durch die Verwendung von Spring, kann dieses sehr einfach implementiert werden:

```
@Bean
public WebMvcConfigurer corsConfigurer() {
    return new WebMvcConfigurerAdapter() {
        @Override
        public void addCorsMappings(CorsRegistry registry) {
            registry.addMapping("/**")
                .allowedOrigins("*")
                .allowedMethods("POST")
                .allowedHeaders("Origin", "Content-Type");
        }
    };
}
```

Abbildung 7: Code für CORS in Spring Boot

Der Code aus Abbildung 7 kann einfach in die „Application“-Klasse eingefügt werden, welche bei jeder Spring-Boot Anwendung generiert wird. Er sorgt dafür, dass für alle Anfragen jegliche Origin erlaubt wird. Bei den erlaubten Methoden ist nur POST aufgelistet, da der Webservice in diesem Fall keine anderen Methoden anbietet. In diesem Fall werden außerdem die Header „Origin“ und „Content-Type“ erlaubt.

4.2. Rest-Service

Der Rest-Service wird, wie schon erwähnt, verwendet um die Berechnungen für CPM und PERT durchzuführen. Dabei wird das Diagramm, welches aktuell in der Webapplikation dargestellt wird, in JSON umgewandelt und in einen AJAX-Request gepackt. Der Rest-Service gibt die deserialisierten Daten dann an den Rest-Controller weiter, welcher dann, abhängig davon, welche URL angesprochen wurde, die Anfrage an den richtigen Controller weiterleitet. In Abbildung 8 wird diese Kommunikation graphisch dargestellt:

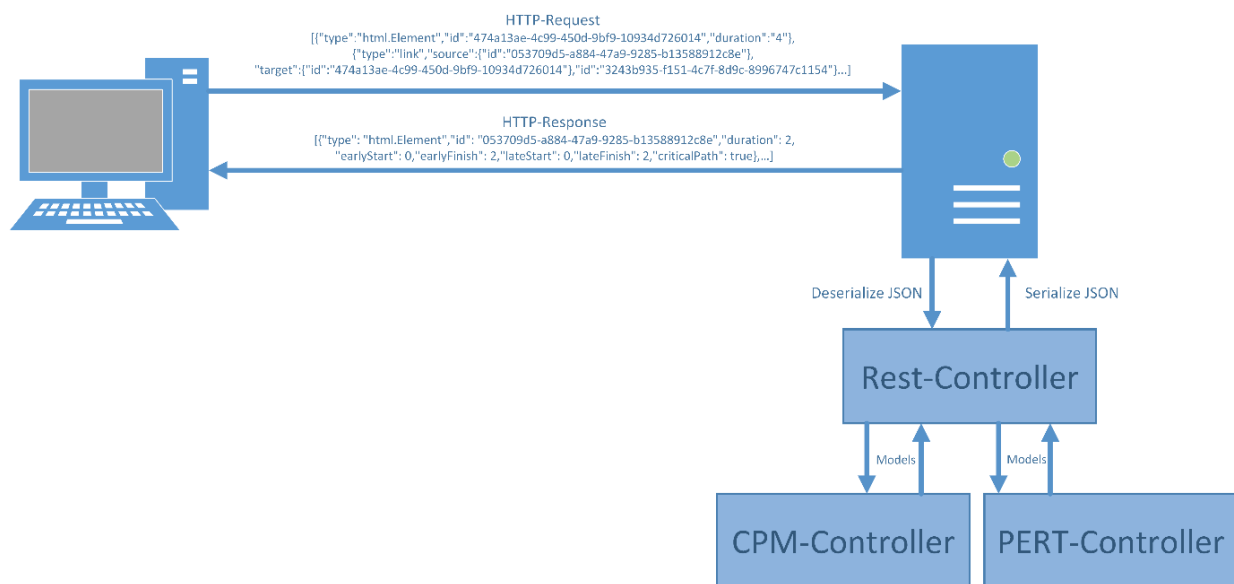


Abbildung 8: Kommunikation mit Rest-Service

1. Der JSON-Text besteht dabei aus einem Array aller Objekte, welche im Diagramm vorhanden sind. Über den Wert „type“ lässt sich anschließend feststellen, ob es sich um eine Aktivität oder eine Verbindung handelt. Im Fall der MC-Simulation wird am Anfang des JSON eine „sampleSize“ mitgesendet. Der JSON-Text kann z. B. folgendermaßen aussehen:

```

[{"type": "sampleSize", "sampleSize": "1000"}, {"type": "html.Element", "inputTitle": "Title1", "minTime": 1, "estTime": 2, "maxTime": 4, "id": "b87804be-a94f-4724-ba1d-e4bf7309207a", "duration": 2, "earlyStart": 0, "lateStart": 0, "earlyFinish": 0, "lateFinish": 0}, {"type": "link", "source": {"id": "b87804be-a94f-4724-ba1d-e4bf7309207a"}, "target": {"id": "0bd3c8e8-8cd7-4247-a4e1-4c492eba52c8"}, "id": "20b9f33e-23bc-4c7b-9233-e8a64d0729de"}, ... ]
  
```

Abbildung 9: JSON Beispiel

Aufbau

Der Server bietet für CPM und PERT jeweils eine Schnittstelle an:

Pfad	Request Content-Type	Response Content-Type	Funktion
/criticalPath	application/json	application/json	Gibt eine überarbeitete Liste von Elementen zurück, welche das früheste und späteste Start- und Enddatum beinhaltet. Ebenfalls gibt ein bool'escher Wert an, ob sich ein Element auf dem kritischen Pfad befindet.
/pert	application/json	application/json	Gibt eine überarbeitete Liste von Elementen zurück, welche eine nach PERT berechnete mittlere Dauer beinhaltet.
/mcsimulation	application/json	application/json	Gibt eine überarbeitete Liste von Elementen zurück, welche eine, mittels der Monte Carlo Simulation, berechnete Dauer beinhaltet. Benötigt eine Zahl, welche angibt, wie oft das Zufallsexperiment durchgeführt werden soll.

Abbildung 10: Liste der Rest-Schnittstellen

5. Umsetzung

5.1. Recherche

Beim Vergleich der verschiedenen Bibliotheken und Frameworks, welche das Zeichnen von Diagrammen ermöglichen, wurden folgende Anforderungen gestellt:

- Elemente müssen mit gerichteten Pfeilen verbunden werden können
- Es müssen eigene Elemente erstellt werden können, je weniger Aufwand desto besser
- Das Framework sollte aktuell sein (neueste Version max. 1 Jahr alt)

React Diagrams erfüllt diese Anforderungen, allerdings stellte es sich als relativ kompliziert heraus eigene Elemente zu erstellen, weshalb anschließend JointJS verwendet wurde, welches ebenfalls alle Anforderungen erfüllt. Für das Erstellen eigener Elemente wurde sogar ein von den Entwicklerinnen erstelltes Tutorial gefunden (<https://resources.jointjs.com/tutorial/custom-elements>), während für React Diagrams keine Beispiele oder ein Tutorial gefunden werden konnten.

Als die Bibliothek für die Webapplikation feststand, wurde nach einer zufriedenstellenden Lösung für den Rest-Service gesucht. Da in vorhergehenden Projekten mit Spring Boot nur positive Erfahrungen gesammelt werden konnten und Spring Boot sich für diesen Einsatzzweck perfekt eignet, musste nicht sehr lange recherchiert werden. Ebenfalls wurden die Auswahlmöglichkeiten durch die Beschränkung auf Java schon zuvor stark eingegrenzt. Alternativen zu Spring Boot wären beispielsweise das ACT.Framework (<http://actframework.org/>) oder Yii (<https://www.yiiframework.com>).

5.2. Rest-Service

Nach der Implementierung des Rest-Service von Spring wurden die verschiedenen Algorithmen umgesetzt werden:

Zuerst wurde der Algorithmus für den kritischen Pfad entwickelt. Es müssen, wie in Kapitel 1.4 erwähnt, alle möglichen Pfade durchlaufen werden. Es werden zuerst alle Startelemente, also alle Elemente, welche nur ausgehende Verbindungen haben, aber keine eingehenden, gesucht. Dann werden ausgehend von diesen Startelementen mittels einer Tiefensuche alle möglichen Pfade durchlaufen. Währenddessen werden das früheste Start- und Enddatum eingetragen und die Gesamtdauer mitgezählt. Alle diese Pfade werden zwischengespeichert, da sie nun in umgekehrter Richtung durchlaufen werden, um das späteste Start- und Enddatum einzutragen. Um den kritischen Pfad festzustellen, kann der Pfad mit der höchsten Gesamtdauer entnommen werden.

Beim Algorithmus zum Berechnen des PERT-Wertes, werden alle Elemente durchlaufen und es wird mittels der Formel aus Kapitel 1.5 die Dauer des Elements berechnet und gespeichert.

Bei der Monte-Carlo-Simulation müssen zuerst für jedes Element alle Werte berechnet werden, welche für die Beta-Verteilung benötigt werden, also Alpha und Beta. Diese werden unter der Verwendung der Minimaldauer, der erwarteten Dauer und der Maximaldauer berechnet. Anschließend werden auf Basis dieser für jedes Element Zufallswerte gezogen und jeweils der kritische Pfad berechnet. Auf Basis dieser Ergebnisse lässt sich dann die Varianz oder die Wahrscheinlichkeit, für jede Aktivität auf dem kritischen Pfad zu liegen, berechnen. Da für jeden Zufallswert bestimmt wird, ob die Aktivität mit diesem auf dem kritischen Pfad liegt, kann so auch eine Wahrscheinlichkeit berechnet werden. Die Varianz kann dabei einfach mit dem Erwartungswert, also dem Mittelwert aller Zufallswerte und den Zufallswerten berechnet werden:

$$\text{Var}(X) := \mathbb{E}((X - \mu)^2)$$

Abbildung 11: Formel Varianz

Der gesamte Ablauf wird in Abbildung 12 als Flussdiagramm dargestellt.

Umsetzung

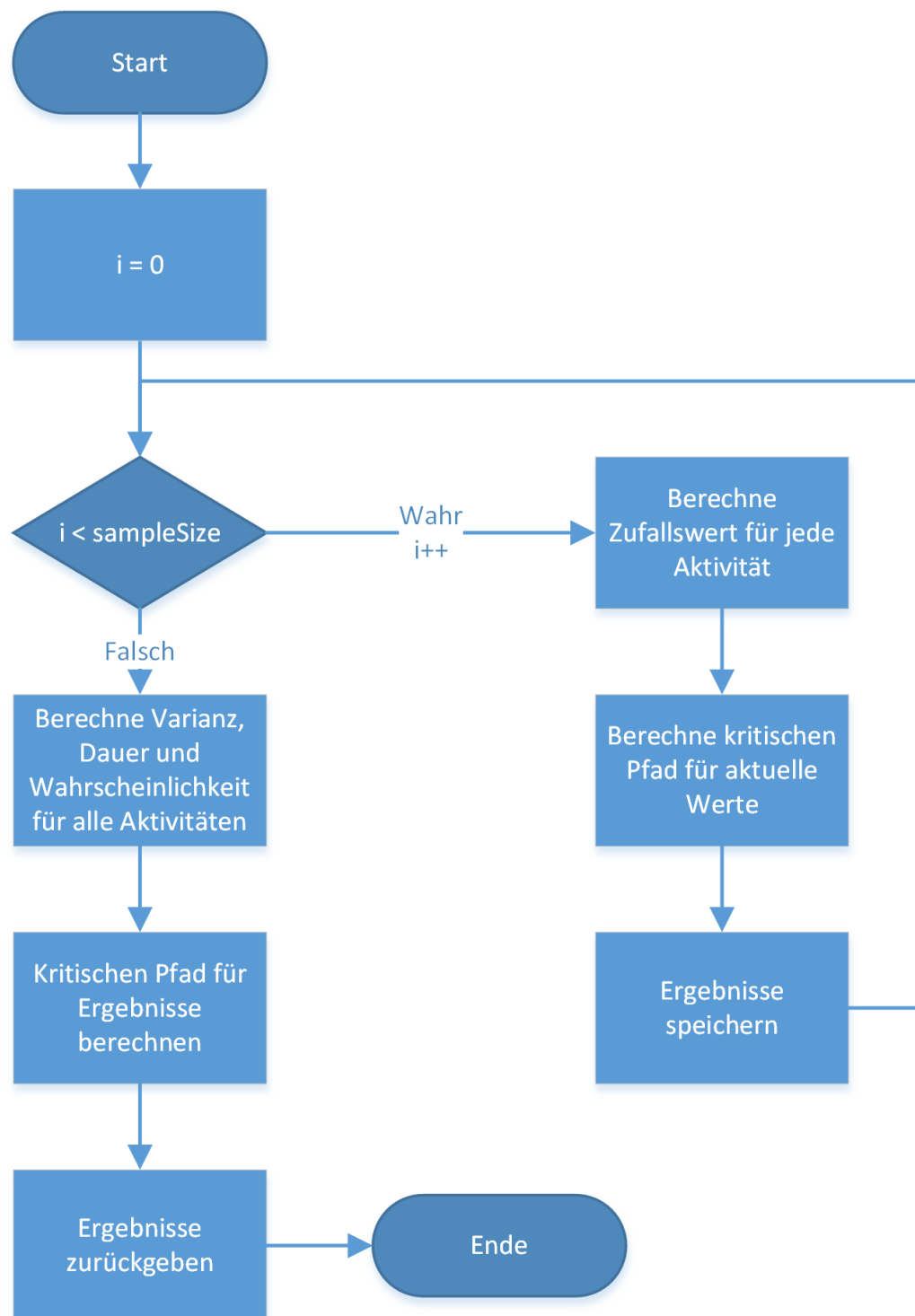


Abbildung 12: Flussdiagramm Monte Carlo Simulation

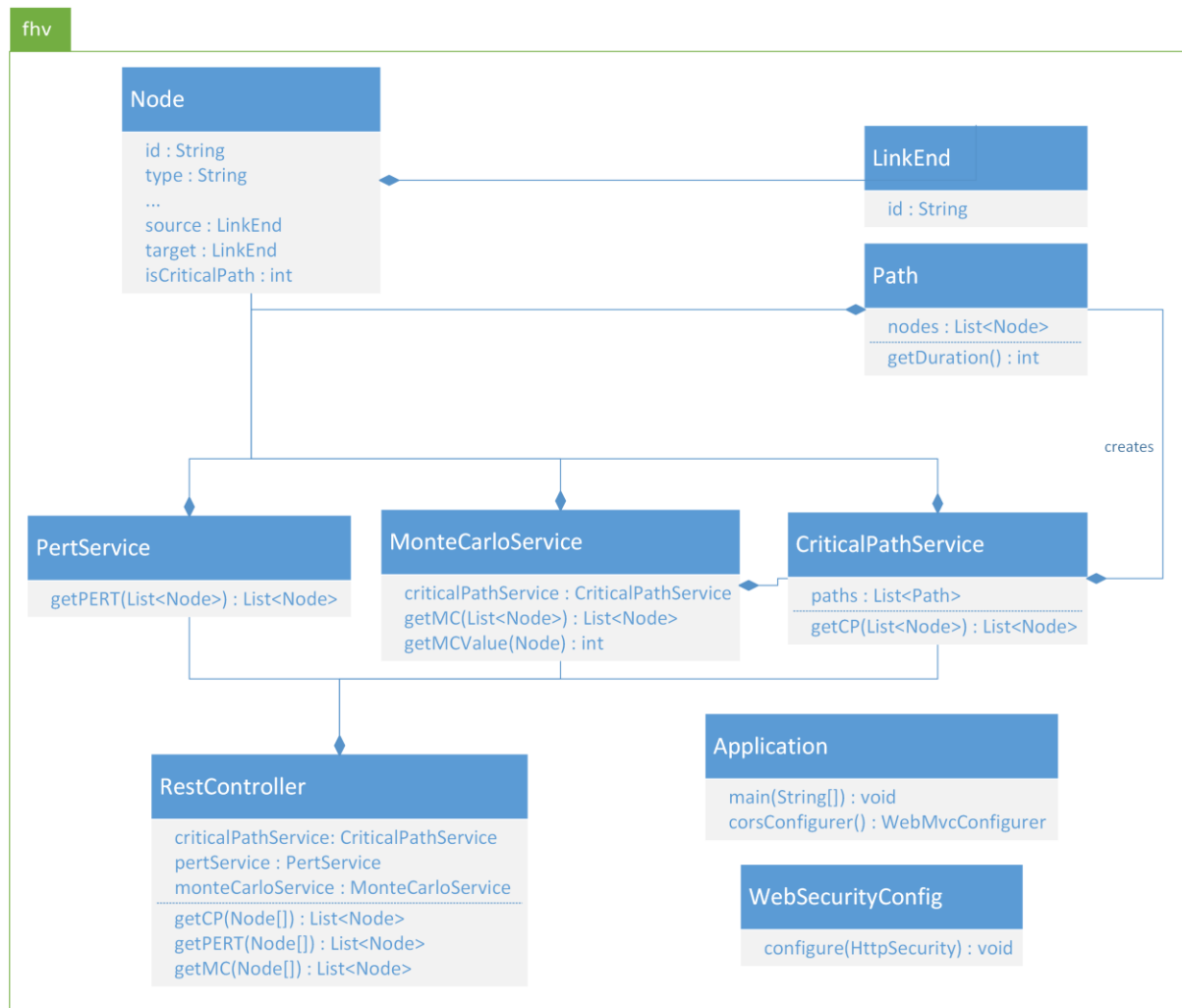


Abbildung 13: Klassendiagramm Rest-Service

5.2.1. RestController

Der RestController erhält bei seinen zwei Methoden jeweils ein Array von Nodes. Dieses Array wandelt er in eine Liste um und ruft, je nach Methode, den dazugehörigen Service auf und gibt die neu erstellte Liste mit. Das Ergebnis des Service gibt er anschließend zurück.

5.2.2. PertService

Dem PertService wird eine Liste von Nodes vom RestController übergeben, welche er anschließend durchläuft und für jedes Element die Dauer berechnet. Die überarbeitete Liste gibt er anschließend wieder an den RestController zurück.

5.2.3. CriticalPathService

Der CriticalPathService bekommt, wie der PertService auch, eine Liste von Nodes. Da diese rekursiv durchlaufen werden, speichert er alle gefundenen Pfade in eine Liste. Mithilfe dieser Liste von Pfaden können anschließend alle Pfade in umgekehrter Richtung durchlaufen werden und es kann dadurch der Pfad mit der maximalen Dauer gefunden werden.

5.2.4. MonteCarloService

Der MonteCarloService erhält, wie die beiden anderen Services auch, eine Liste von Nodes. Anschließend wird für jedes Element mit der Funktion `getMCValue()` ein Zufallswert berechnet, um dann mit Hilfe des CriticalPathService den kritischen Pfad für die aktuellen Zufallswerte zu berechnen. Dies wird X-Mal wiederholt, wobei alle Ergebnisse gespeichert werden, um am Ende die Ergebnisse berechnen zu können, welche in den Elementen gespeichert werden.

5.2.5. Path

Der Path ist eine Klasse, welche eine Liste von Nodes beinhaltet, welche miteinander verbunden sind. Mithilfe der Methode `getDuration()` kann der CriticalPathService einfach die Dauer des Pfades abfragen.

5.2.6. Node

Die Klasse Node ist eine einfache Klasse, welche nur Daten beinhaltet. Das Attribut `type` wird benötigt um festzustellen, ob es sich um ein Element oder eine Verbindung handelt. Nur wenn es sich um eine Verbindung handelt werden die Attribute `source` und `target` gesetzt bzw. verwendet. Zusätzlich zu den im Diagramm dargestellten Attributen beinhaltet die Node noch weitere Attribute für alle Zeitangaben (`minTime`, `estTime`, `maxTime`, `duration`, `earlyStart`, `lateStart`, `earlyFinish`, `lateFinish`), welche alle als `int` gespeichert werden.

5.2.7. LinkEnd

Diese Klasse wird benötigt, damit die Struktur der Klassen mit der Struktur des JSON übereinstimmt, welches von JointJS demensprechend generiert wird. Die Klasse beinhaltet nur ein Attribut, welches entweder die ID des Start- oder des Endelementes angibt.

5.2.8. Application

Die Applikation Klasse wird automatisch beim Erstellen der Spring-Boot Anwendung generiert. Diese Klasse wird kaum verändert, es wurde nur der Code für CORS, wie in Kapitel 4.1.1 erwähnt, hinzugefügt.

5.2.9. WebSecurityConfig

Diese Klasse wird benötigt, um die Sicherheitseinstellungen der Spring-Boot Anwendung zu definieren. Sie wurde dahingehend angepasst, dass jeder User ohne Authentifizierung die Anwendung verwenden kann:

```
@Configuration
@EnableWebSecurity
public class WebSecurityConfig extends WebSecurityConfigurerAdapter {

    @Override
    protected void configure(HttpSecurity http) throws Exception {
        http
            .authorizeRequests()
            .anyRequest().permitAll();

        http.csrf().disable();
    }
}
```

Abbildung 14: WebSecurityConfig Klasse

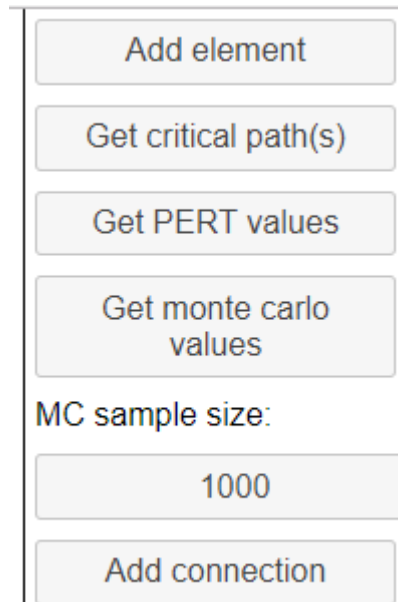
Es wird dabei, wie in Abbildung 14 ersichtlich, jede Anfrage erlaubt und auch die Cross-Site-Request-Forgery (CSRF) deaktiviert, da diese standardmäßig Anfragen von anderen Seiten, also in diesem Fall von unserer Webanwendung, blockiert. Ohne diesen Zusatz würden die Anfragen der Webanwendung sofort abgelehnt werden.

5.3. Webapplikation

Da ein Tutorial gefunden wurde, welches alle benötigten Schritte beinhaltet, welche notwendig sind um ein eigenes Element zu erstellen, wurde als erstes dieses Tutorial Schritt für Schritt abgearbeitet. Anschließend wurde das Element auf die eigenen Bedürfnisse angepasst.

In Abbildung 15 werden alle Buttons der Webanwendung dargestellt. Der Button „Add element“ kann verwendet werden, um ein neues Element in den Diagrammbereich einzufügen. Damit man die eingefügten Elemente auch verbinden kann, gibt es den Button „Add connection“. Wird dieser angeklickt, können anschließend die zwei Elemente, welche verbunden werden sollen, angeklickt werden, um eine Verbindung einzufügen.

Um die Funktionen des Rest-Services in Anspruch zu nehmen, wurden die Buttons „Get critical path(s)“, „Get PERT values“ und „Get monte carlo values“ eingefügt.



The image shows a vertical stack of five light gray buttons with rounded corners. The buttons are labeled: 'Add element', 'Get critical path(s)', 'Get PERT values', 'Get monte carlo values', and 'Add connection'. Below the buttons, the text 'MC sample size:' is displayed in a blue font, followed by a light gray text input field containing the number '1000'.

Abbildung 15: Buttons der Webanwendung

Mit „Get PERT values“ sendet die Webanwendung die eingetragenen Werte Mindestdauer, erwartete Dauer und Maximaldauer für jedes Element an den Rest-Service. Die daraus berechnete Dauer wird dann in das Diagramm eingetragen.

Die Funktion „Get monte carlo values“ berechnet, unter Verwendung von Zufallsexperimenten, die Dauer, die Wahrscheinlichkeit mit welcher ein Element auf dem kritischen Pfad liegt und eine Varianz. Unterhalb kann die Anzahl an durchgeführten Zufallsexperimenten definiert werden. Je nach Diagrammgröße kann diese Funktion unter Umständen mehrere Sekunden in Anspruch nehmen.

Umsetzung

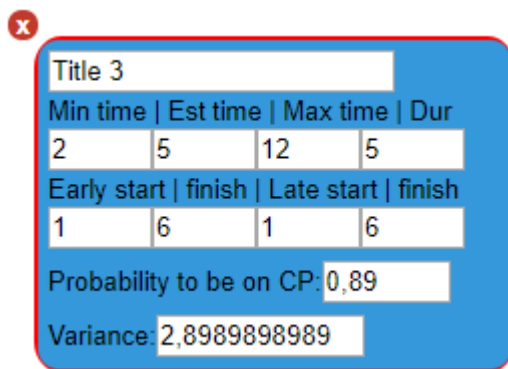
Um den bzw. alle kritischen Pfade zu finden, kann der Button „Get critical path(s)“ verwendet werden. Dabei werden alle Elemente und Verbindungen des Diagramms an den Server gesendet, welcher diese anschließend, wie in Kapitel 5.2 erklärt, analysiert. Das Ergebnis muss anschließend analysiert werden, wobei alle Elemente, welche auf dem kritischen Pfad liegen, rot umrandet werden. Ebenfalls müssen in jedes Elemente das früheste und das späteste Start- und Enddatum eingetragen werden. Da das Aussehen durch Einstellungen von JointJS, aber auch durch Werte in den CSS-Dateien beeinflusst wird, müssen einige Anpassungen vorgenommen werden, damit die Elemente, welche auf dem kritischen Pfad liegen, auch rot umrandet werden können bzw. damit diese rote Umrandung auch sichtbar ist.

6. Ergebnisse

Das Ziel dieser Arbeit war, eine Webapplikation zu entwickeln, welche mit Hilfe eines Rest-Services Analysen wie CPM, PERT und die MC-Simulation durchführen kann. Dabei wurden die genauen Anforderungen in Kapitel 0 definiert. In diesem Kapitel werden die gestellten Anforderungen auf ihre Erfüllung überprüft.

6.1. Webapplikation

Die Webapplikation ermöglicht das Modellieren von PDM-Netzwerken. Mittels eines Buttons unter dem Diagrammbereich, können neue Elemente erstellt werden. Um eine neue Verbindung einzufügen, kann einfach das Element von dem die Verbindung ausgehen soll, doppelt angeklickt werden. Damit alle benötigten Daten eingegeben werden können, wurde ein eigenes Element erstellt (Abbildung 16):



Title 3			
Min time	Est time	Max time	Dur
2	5	12	5
Early start	finish	Late start	finish
1	6	1	6
Probability to be on CP: 0,89			
Variance: 2,8989898989			

Abbildung 16: Eigenes Element

Werte wie die Minimaldauer, die erwartete Dauer und die Maximaldauer müssen dabei vom User eingegeben werden. Alternativ kann der User auch direkt die Dauer eintragen, z.B., wenn man sich sicher ist, wie lange etwas dauert oder wenn es schon abgeschlossen wurde. Die Werte der zweiten Zeile werden durch die CP-Methode oder die MC-Simulation berechnet und können nicht vom User eingegeben werden. Die Wahrscheinlichkeit für ein Element auf dem kritischen Pfad zu liegen und die Varianz werden ebenfalls mittels der MC-Simulation berechnet und erlauben keine Benutzereingaben.

Ergebnisse

Um die Services des Rest-Service in Anspruch nehmen zu können wurden zwei Buttons eingefügt. Einer, um den kritischen Pfad berechnen zu lassen und ein anderer, um mittels PERT die Dauer auszurechnen. Dabei benötigt PERT zum einen Angaben zur minimalen, erwarteten und maximalen Dauer und zum anderen, um den kritischen Pfad berechnen zu können, müssen alle Elemente über eine Dauer verfügen und korrekt miteinander verbunden sein. Sollte das Diagramm nicht korrekt verbunden sein, wird dem User eine Fehlermeldung angezeigt. Wenn z.B. keine Startelemente gefunden werden, wird folgende Meldung angezeigt (Abbildung 17):

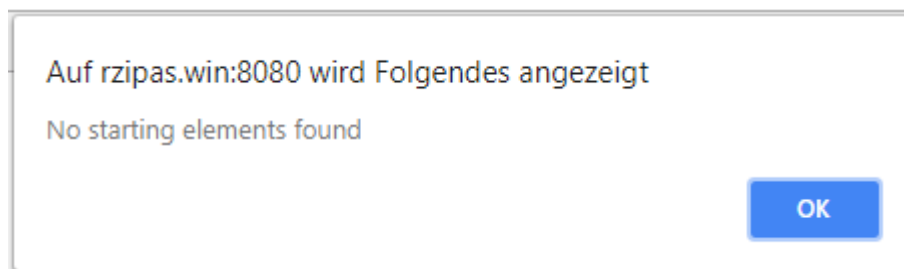


Abbildung 17: Fehlermeldung – Keine Startelemente gefunden

Sollte ein Zyklus im Diagramm vorhanden sein, wird folgende Meldung angezeigt:

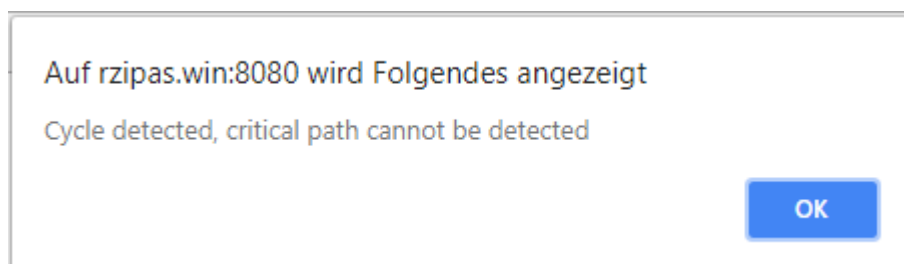


Abbildung 18: Fehlermeldung - Zyklus entdeckt

Ergebnisse

Wurde die Berechnung des kritischen Pfades erfolgreich beendet, so wird das Ergebnis visuell, wie in Abbildung 19 gezeigt, dargestellt.

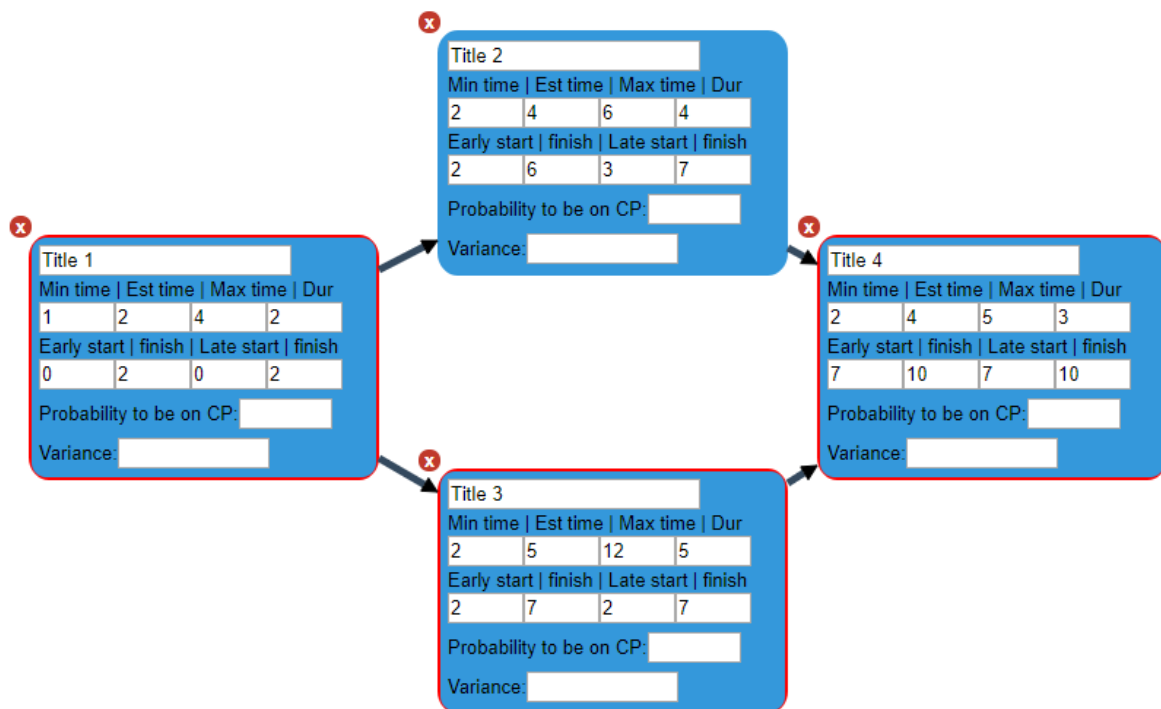


Abbildung 19: Diagramm mit kritischem Pfad

Dabei werden alle Elemente, welche auf dem kritischen Pfad liegen, rot umrandet. Ebenfalls werden die frühesten und die spätesten Start- und Endzeiten für alle Elemente eingetragen.

Das Ergebnis der Webapplikation umfasst eine HTML-Datei, drei JavaScript-Dateien und eine CSS-Datei für JointJS, eine JavaScript-Datei für JQuery und eine CSS-Datei für das eigens erstellte Element. Um die Webapplikation starten zu können, können diese Dateien einfach auf einen beliebigen Webservice kopiert werden.

Die Applikation funktioniert in jedem modernen Browser (z.B.: Chrome, Edge, Safari, Firefox) und sollte dabei auch keinerlei Kompatibilitätsprobleme aufweisen.

6.2. Rest-Service

Der Rest-Service bietet alle in Kapitel 4.2 erwähnten Funktionen an. Dabei wäre es möglich, dass der Rest-Service auch von anderen Anwendungen verwendet werden kann, sofern diese dieselbe Struktur für die JSON-Daten verwenden. Dank der Implementierung von CORS könnten diese Anwendungen auch von verschiedenen Domänen aus auf den Rest-Service zugreifen.

Der Rest-Service wurde mittels Maven in eine JAR-Datei verpackt, welche auf jedem Java-fähigem Gerät gestartet werden kann, dabei wird Standardmäßig der Port 8090 verwendet.

6.3. Ausblick/Erkenntnisse

Dank der Verwendung von Spring, wäre es ohne Umstände möglich, den Server um weitere Funktionen zu erweitern. Sollte der Server in eine bestehende Umgebung, welche auf Docker Containern basiert, eingebunden werden, so wäre dies dank der Verwendung von Spring ebenfalls leicht möglich. Es wäre auch denkbar, weitere Funktionen zur Analyse des PDM-Netzwerkes in dem Rest-Service einzubinden. Mit Hilfe von Spring müsste hierfür eine weitere Funktion eingefügt werden und es müsste ausschließlich diese implementiert werden. Bei der Webapplikation müsste dann ein weiterer Button eingefügt werden, um den neu hinzugefügten Service verwenden zu können. Hier müsste nur die Auswertung des Ergebnisses vom Rest-Service implementiert werden.

7. Literaturverzeichnis

„Building a RESTful Web Service“ (o. J.): Building a RESTful Web Service. Online im Internet: URL: <https://spring.io/guides/gs/rest-service/> (Zugriff am: 23.04.2018).

DavidJWilliamson (2016): David J. Williamson, PhD, PMP, ITIL. Better Project Management Through Beta Distribution. Online im Internet: URL: <https://davidjwilliamson.net/2016/11/02/better-project-management-through-beta-distribution/> (Zugriff am: 16.05.2018).

„GanttProject: free desktop project management app“ (o. J.): GanttProject: free desktop project management app. Online im Internet: URL: <https://www.ganttproject.biz/> (Zugriff am: 18.05.2018).

Haverbeke, Marijn (2015): Eloquent JavaScript: A Modern Introduction to Programming. 2 edition. San Francisco: No Starch Press.

„Home : The Official Microsoft IIS Site“ (o. J.): Home : The Official Microsoft IIS Site. Online im Internet: URL: <https://www.iis.net/> (Zugriff am: 16.05.2018).

„Introduction · react-diagrams“ (o. J.): Introduction · react-diagrams. Online im Internet: URL: <https://projectstorm.gitbooks.io/react-diagrams/> (Zugriff am: 23.04.2018).

„JointJS: Visualize and interact with diagrams and graphs“ (o. J.): JointJS: Visualize and interact with diagrams and graphs. Online im Internet: URL: <https://www.jointjs.com/opensource> (Zugriff am: 23.04.2018).

jquery.org, jQuery Foundation- (o. J.): jQuery. Online im Internet: URL: <https://jquery.com/> (Zugriff am: 23.04.2018).

Liquidplanner, Team (o. J.): LiquidPlanner. LiquidPlanner: Online Project Management Software for Your Business. Online im Internet: URL: <https://www.liquidplanner.com/> (Zugriff am: 18.05.2018).

Nahmias, Steven; Olsen, Tava Lennon (2015): Production and Operations Analysis: Strategy - Quality - Analytics - Application. 7 edition. Long Grove, Ill: Waveland Pr Inc.

Niemann, Alexander (2010): Objektorientierte Programmierung in Java. 6 edition. Heidelberg Hamburg: Verlagsgruppe Hüthig-Jehle-Rehm.

„Online Diagram Software & Visual Solution“ (2018): Lucidchart. Online Diagram Software & Visual Solution. Online im Internet: URL: <https://www.lucidchart.com> (Zugriff am: 23.04.2018).

„PDM-1-768x328.jpg (768×328)“ (o. J.): PDM-1-768x328.jpg (768×328). Online im Internet: URL: <http://cemsolutions.org/wp-content/uploads/2016/06/PDM-1-768x328.jpg> (Zugriff am: 18.04.2018).

Pilgrim, Mark (2010): HTML5: Up and Running. 1 edition. Beijing: O'Reilly and Associates.

„Postman“ (o. J.): Postman. Postman. Online im Internet: URL: <https://www.getpostman.com/> (Zugriff am: 01.05.2018).

„react-diagrams: a super simple, no-nonsense diagramming library written in react that just works“ (2018): react-diagrams: a super simple, no-nonsense diagramming library written in react that just works. Storm. Online im Internet: URL: <https://github.com/projectstorm/react-diagrams> (Zugriff am: 23.04.2018).

Rubinstein, Reuven Y.; Kroese, Dirk P. (2007): Simulation and the Monte Carlo Method. 2 edition. Hoboken, N.J: Wiley-Interscience.

„Spring Boot Development with Docker“ (2017): Docker Blog. Spring Boot Development with Docker. Online im Internet: URL: <https://blog.docker.com/2017/05/spring-boot-development-docker/> (Zugriff am: 16.05.2018).

Tilkov, Stefan u.a. (2015): REST und HTTP: Entwicklung und Integration nach dem Architekturstil des Web. 3., akt. u. erw. Aufl. edition. Heidelberg: dpunkt.verlag GmbH.

United States; Bureau of Naval Weapons; Special Projects Office (1958): Program evaluation research task (PERT) summary report; phase 1. Washington: Special Projects Office, Bureau of Naval Weapons, Dept. of the Navy.

„XAMPP Installers and Downloads for Apache Friends“ (o. J.): XAMPP Installers and Downloads for Apache Friends. Online im Internet: URL: <https://www.apachefriends.org/de/index.html> (Zugriff am: 16.05.2018).

8. Abbildungsverzeichnis

Abbildung 1: PDM-Netzwerk („PDM-1-768x328.jpg (768×328)“ o. J.).....	3
Abbildung 2: Beispiel CPM	5
Abbildung 3: Formel für mittlere Dauer	6
Abbildung 4: Beta-Verteilung (DavidJWilliamson 2016).....	6
Abbildung 5: Benötigte Werte für PERT und Ergebnis	6
Abbildung 6: Client-Server Architektur.....	13
Abbildung 7: Code für CORS in Spring Boot.....	14
Abbildung 8: Kommunikation mit Rest-Service	15
Abbildung 9: JSON Beispiel	15
Abbildung 10: Liste der Rest-Schnittstellen	16
Abbildung 11: Formel Varianz.....	18
Abbildung 12: Flussdiagramm Monte Carlo Simulation	19
Abbildung 13: Klassendiagramm Rest-Service.....	20
Abbildung 14: WebSecurityConfig Klasse	22
Abbildung 15: Buttons der Webanwendung	23
Abbildung 16: Eigenes Element	25
Abbildung 17: Fehlermeldung – Keine Startelemente gefunden	26
Abbildung 18: Fehlermeldung - Zyklus entdeckt	26
Abbildung 19: Diagramm mit kritischem Pfad	27